Student Name	Sarvesh Naik
SRN No	31241943
Roll No	33
Program	Computer Engineering
Year	Third Year
Division	F
Subject	Artificial Intelligence Lab
Assignment No	7

Assignment 7

Problem Statement: Implement the Minimax algorithm to solve the Tic Tac Toe problem

Objective:

The objective of this assignment is to implement the Minimax algorithm, an artificial intelligence technique used in decision-making and game playing. The goal is to enable a computer (AI player) to play Tic Tac Toe optimally against an opponent by evaluating all possible future moves and selecting the best one.

This assignment also aims to:

- Understand adversarial search techniques.
- Learn how recursive decision-making works.
- Apply game tree concepts in solving real-world problems.
- Explore optimization using Alpha-Beta Pruning.

Theory:

The **Minimax Algorithm** is a decision-making algorithm used in **Artificial Intelligence**, particularly in two-player, turn-based games such as Tic Tac Toe, Chess, and Checkers. It is used to determine the optimal move for a player assuming that the opponent is also playing optimally.

1. Basic Concept

Minimax works on the principle of:

- **Maximizing the player's advantage**
- **Minimizing the opponent's advantage**

There are two types of players:

- **Maximizing Player (MAX)** → tries to maximize the score
- **Minimizing Player (MIN)** → tries to minimize the score

In Tic Tac Toe:

- Player **X** → **Maximizer**
- Player **O** → **Minimizer**

2. Game Tree Representation

The algorithm represents the game as a **tree structure**, where:

- Each **node** represents a board state
- Each **edge** represents a possible move
- The **root node** is the current game state
- The **leaf nodes** represent terminal states (win, loss, draw)

The algorithm explores all possible future moves until it reaches terminal states.

3. Evaluation Function (Utility Function)

To determine the best move, Minimax uses a scoring system:

- If **MAX (X) wins** → Score = +10
- If **MIN (O) wins** → Score = -10
- If **Draw** → Score = 0

To prefer faster wins and delay losses, depth is considered:

- Score = 10 - depth (for MAX win)
- Score = depth - 10 (for MIN win)

Working:

The Minimax algorithm works on the principle of **maximizing the AI's chances of winning while minimizing the opponent's chances**.

Key Concepts:

1. **Players:**
    - **Maximizing Player (X):** AI tries to maximize the score.
    - **Minimizing Player (O):** Opponent tries to minimize the score.
2. **Game Tree:**
    - The algorithm explores all possible game states (moves).
    - Each node represents a board configuration.
3. **Terminal States:**
    - Win → Score assigned (+10 or -10)
    - Draw → Score = 0
4. **Recursive Evaluation:**

    - The algorithm explores all possible moves using recursion.
    - It assigns scores to each move based on outcomes.
5. **Backtracking:**
    - After exploring a move, it backtracks to evaluate other possibilities.
6. **Optimal Move Selection:**
    - The best move is chosen based on:
        - Maximum score for AI (X)
        - Minimum score for opponent (O)

Algorithm:

Start

Initialize the Board

- Represent the Tic Tac Toe board as a list or array of 9 cells.

Check Terminal State

- If a player (X or O) has won → return score
- If board is full (draw) → return 0

Define Scoring Function

- If **X (Maximizer) wins** → return **+10 − depth**
- If **O (Minimizer) wins** → return **depth − 10**
- If **Draw** → return **0**

Recursive Minimax Function

- Input: board, depth, isMaximizing

If Maximizing Player (X):

- Set bestScore = −∞
- For each empty cell:
    - Place 'X' in the cell
    - Recursively call minimax for minimizing player
    - Undo the move (backtrack)
    - Update bestScore = max(bestScore, returned value)
- Return bestScore

If Minimizing Player (O):

- Set bestScore = +∞
- For each empty cell:
    - Place 'O' in the cell
    - Recursively call minimax for maximizing player
    - Undo the move (backtrack)
    - Update bestScore = min(bestScore, returned value)
- Return bestScore

Find Best Move

- For all possible moves:
    - Apply move for 'X'
    - Call minimax
    - Choose move with highest score

Apply Alpha-Beta Pruning (Optimization)

- Maintain two values:
    - Alpha (α) = best value for maximizer
    - Beta (β) = best value for minimizer
- While exploring:
    - If **β ≤ α**, stop exploring further (prune branch)

Return Best Move

End.

Pseudo Code:

```
function MINIMAX(state, depth, isMaximizing):


  if terminal state:

    return score of state


  if isMaximizing:

    best = -∞

    for each move in state:

      value = MINIMAX(resulting_state, depth+1, false)

      best = max(best, value)

    return best


  else:

    best = +∞

    for each move in state:

      value = MINIMAX(resulting_state, depth+1, true)
```

```
            best = min(best, value)
        return best
function MINIMAX(state, depth, isMaximizing, alpha, beta):

    if terminal state:
        return score

    if isMaximizing:
        best = -∞
        for each move:
            value = MINIMAX(child, depth+1, false, alpha, beta)
            best = max(best, value)
            alpha = max(alpha, best)
            if beta ≤ alpha:
                break
        return best

    else:
        best = +∞
        for each move:
            value = MINIMAX(child, depth+1, true, alpha, beta)
            best = min(best, value)
            beta = min(beta, best)
            if beta ≤ alpha:
                break
        return best
```

Code:

```
"""
Minimax Algorithm - Tic Tac Toe
AI Assignment Submission
"""


import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch, Circle
import matplotlib.patheffects as pe
import matplotlib.patches as mpatches
from matplotlib.lines import Line2D
import numpy as np
import math
import copy

import os

os.makedirs("outputs-7", exist_ok=True)

# ─────────────────────────────────────────────────────────────
#  COLOUR PALETTE
# ─────────────────────────────────────────────────────────────
BG       = "#0D1117"
PANEL    = "#161B22"
```

```
BORDER    = "#30363D"
ACCENT_X = "#FF6B6B"    # X player (Maximiser) – coral-red
ACCENT_O = "#4ECDC4"    # O player (Minimiser) – teal
GOLD      = "#FFD166"    # best move highlight
WHITE     = "#E6EDF3"
GREY      = "#8B949E"
GREEN     = "#3FB950"
PURPLE    = "#BC8CFF"
ORANGE    = "#FFA657"


plt.rcParams.update({
    "font.family": "monospace",
    "text.color":  WHITE,
    "axes.facecolor": BG,
    "figure.facecolor": BG,
})


# ═══════════════════════════════════════════════════════════
#  CORE MINIMAX ENGINE
# ═══════════════════════════════════════════════════════════


def check_winner(board):
    lines = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    for a,b,c in lines:
        if board[a] == board[b] == board[c] != ' ':
```

```
            return board[a], (a,b,c)
    return None, None

def is_terminal(board):
    winner, _ = check_winner(board)
    return winner is not None or ' ' not in board

def get_available_moves(board):
    return [i for i,v in enumerate(board) if v == ' ']

def minimax(board, depth, is_maximising, alpha=-math.inf, beta=math.inf,
            tree_data=None, parent_id=None, move_made=None):
    """
    Minimax with Alpha-Beta Pruning.
    Optionally records the search tree into `tree_data` (a list of node dicts).
    """
    node_id = len(tree_data) if tree_data is not None else None

    winner, _ = check_winner(board)
    if winner == 'X':  score = 10 - depth
    elif winner == 'O': score = depth - 10
    elif ' ' not in board: score = 0
    else: score = None  # not terminal

    if tree_data is not None:
```

```
    node = {
        "id":         node_id,
        "parent":     parent_id,
        "board":      board[:],
        "depth":      depth,
        "is_max":     is_maximising,
        "score":      score,
        "move":       move_made,
        "pruned":     False,
        "alpha":      alpha,
        "beta":       beta,
    }
    tree_data.append(node)


if score is not None:
    return score


moves = get_available_moves(board)
best_score = -math.inf if is_maximising else math.inf
best_move  = None


for mv in moves:
    new_board = board[:]
    new_board[mv] = 'X' if is_maximising else 'O'
    s = minimax(new_board, depth + 1, not is_maximising,
```

```
                    alpha, beta, tree_data, node_id, mv)


        if is_maximising:
            if s > best_score:
                best_score, best_move = s, mv
            alpha = max(alpha, s)
        else:
            if s < best_score:
                best_score, best_move = s, mv
            beta = min(beta, s)

        if beta <= alpha:
            if tree_data is not None:
                tree_data[node_id]["pruned"] = True
            break

    if tree_data is not None:
        tree_data[node_id]["score"] = best_score
        tree_data[node_id]["best_move"] = best_move

    return best_score


def best_move(board):
    """Return the best move index for 'X' (maximiser)."""
    best_sc = -math.inf
```

```
    bm = None
    for mv in get_available_moves(board):
        b = board[:]
        b[mv] = 'X'
        s = minimax(b, 0, False)
        if s > best_sc:
            best_sc, bm = s, mv
    return bm, best_sc




# ═══════════════════════════════════════════════════════════════
#  HELPER – draw a single board cell grid
# ═══════════════════════════════════════════════════════════════


def draw_board_on_ax(ax, board, highlight_cells=None, win_line=None,
                     title="", title_color=WHITE, fontsize=7, cell_size=1):
    ax.set_xlim(0, 3*cell_size); ax.set_ylim(0, 3*cell_size)
    ax.set_aspect('equal')
    ax.axis('off')
    if title:
        ax.set_title(title, color=title_color, fontsize=fontsize,
                     fontweight='bold', pad=2)

    hc = set(highlight_cells) if highlight_cells else set()
    wl = set(win_line) if win_line else set()
```

```
for idx in range(9):
    row, col = divmod(idx, 3)
    y = (2 - row) * cell_size
    x = col * cell_size

    fc = "#1C2733" if idx in hc else ("#0D2818" if idx in wl else PANEL)
    ec = GOLD if idx in hc else (GREEN if idx in wl else BORDER)
    lw = 1.8 if (idx in hc or idx in wl) else 0.6

    rect = FancyBboxPatch((x+0.04*cell_size, y+0.04*cell_size),
                          0.92*cell_size, 0.92*cell_size,
                          boxstyle="round,pad=0.02",
                          facecolor=fc, edgecolor=ec, linewidth=lw)
    ax.add_patch(rect)

    val = board[idx]
    if val != ' ':
        clr = ACCENT_X if val == 'X' else ACCENT_O
        ax.text(x + 0.5*cell_size, y + 0.5*cell_size, val,
                ha='center', va='center',
                fontsize=fontsize*1.8, color=clr,
                fontweight='black',
                path_effects=[pe.withStroke(linewidth=1, foreground=BG)])
```

```
# ═══════════════════════════════════════════════════════════════
#  FIGURE 1 – ALGORITHM OVERVIEW  (2-page explainer)
# ═══════════════════════════════════════════════════════════════


def fig_overview():
    fig = plt.figure(figsize=(18, 10), facecolor=BG)
    fig.suptitle("MINIMAX ALGORITHM  —  Tic Tac Toe",
                 fontsize=22, fontweight='black', color=WHITE, y=0.97)

    gs = gridspec.GridSpec(2, 4, figure=fig, hspace=0.55, wspace=0.35,
                           left=0.04, right=0.96, top=0.90, bottom=0.05)

    # ── Left column: pseudocode ──────────────────────────────
    ax_code = fig.add_subplot(gs[:, 0])
    ax_code.set_facecolor(PANEL)
    ax_code.axis('off')
    ax_code.set_title("Pseudocode", color=GOLD, fontsize=11, fontweight='bold')

    code = (
        "function MINIMAX(state, depth,\n"
        "                 isMaximising):\n\n"
        "  if TERMINAL(state):\n"
        "    return SCORE(state, depth)\n\n"
        "  if isMaximising:\n"
```

```
    "    best ← -∞\n"
    "    for move in MOVES(state):\n"
    "      val ← MINIMAX(APPLY(state,move),\n"
    "                    depth+1, FALSE)\n"
    "      best ← MAX(best, val)\n"
    "    return best\n\n"
    "  else:  # minimising\n"
    "    best ← +∞\n"
    "    for move in MOVES(state):\n"
    "      val ← MINIMAX(APPLY(state,move),\n"
    "                    depth+1, TRUE)\n"
    "      best ← MIN(best, val)\n"
    "    return best"
)
ax_code.text(0.05, 0.95, code, transform=ax_code.transAxes,
             fontsize=8.5, color=WHITE, va='top', fontfamily='monospace',
             linespacing=1.55)

# colour-coded legend for pseudocode
for y_, txt, clr in [(0.06, "■  Maximiser (X)", ACCENT_X),
                     (0.03, "■  Minimiser (O)", ACCENT_O)]:
    ax_code.text(0.05, y_, txt, transform=ax_code.transAxes,
                 fontsize=8, color=clr)

# ── Scoring rules ──────────────────────────────────────────
```

```
ax_score = fig.add_subplot(gs[0, 1])
ax_score.set_facecolor(PANEL); ax_score.axis('off')
ax_score.set_title("Scoring Function", color=GOLD, fontsize=11, fontweight='bold')
rules = [
    ("X wins",   "+10 - depth", ACCENT_X),
    ("O wins",   "depth - 10",  ACCENT_O),
    ("Draw",     "0",           GREY),
    ("X plays",  "Maximise ↑",  ACCENT_X),
    ("O plays",  "Minimise ↓",  ACCENT_O),
]
for i,(lbl,val,clr) in enumerate(rules):
    y_pos = 0.82 - i*0.16
    ax_score.add_patch(FancyBboxPatch((0.04, y_pos-0.06), 0.92, 0.12,
                       boxstyle="round,pad=0.01",
                       facecolor=BG, edgecolor=clr, linewidth=1.2,
                       transform=ax_score.transAxes))
    ax_score.text(0.10, y_pos, lbl, transform=ax_score.transAxes,
                  fontsize=9, color=clr, va='center', fontweight='bold')
    ax_score.text(0.90, y_pos, val, transform=ax_score.transAxes,
                  fontsize=9, color=WHITE, va='center', ha='right')

# ── Alpha-Beta box ──────────────────────────────────────
ax_ab = fig.add_subplot(gs[1, 1])
ax_ab.set_facecolor(PANEL); ax_ab.axis('off')
ax_ab.set_title("Alpha-Beta Pruning", color=GOLD, fontsize=11, fontweight='bold')
```

```
ab_text = (
    "Improvement over plain Minimax.\n\n"
    "α  = best score Maximiser can guarantee\n"
    "β  = best score Minimiser can guarantee\n\n"
    "Prune when  β ≤ α\n"
    "(remaining branches can't affect result)\n\n"
    "Complexity:\n"
    "  Plain Minimax:  O(b^m)\n"
    "  Alpha-Beta:     O(b^(m/2))  (best case)\n\n"
    "For Tic-Tac-Toe:\n"
    "  b ≈ 5,  m = 9  →  255,168 positions\n"
    "  With pruning: significantly fewer nodes"
)
ax_ab.text(0.05, 0.92, ab_text, transform=ax_ab.transAxes,
           fontsize=8.5, color=WHITE, va='top', linespacing=1.5)

# ── Example position: X to move ──────────────────────
example_board = ['X', ' ', 'O',
                 ' ', 'X', ' ',
                 'O', ' ', ' ']
bm_idx, bm_sc = best_move(example_board)

ax_ex = fig.add_subplot(gs[0, 2])
draw_board_on_ax(ax_ex, example_board,
                 highlight_cells=[bm_idx],
```

```
                    title=f"Example: X to move  →  best={bm_sc}",
                    title_color=GOLD, fontsize=9)

    # ── After best move ──────────────────────────────────────
    after = example_board[:]
    after[bm_idx] = 'X'
    winner, wl = check_winner(after)
    ax_af = fig.add_subplot(gs[1, 2])
    draw_board_on_ax(ax_af, after,
                    win_line=list(wl) if wl else None,
                    title="After best move  →  X wins!" if winner == 'X' else "After
best move",
                    title_color=GREEN if winner else WHITE, fontsize=9)

    # ── Game-tree summary ────────────────────────────────────
    ax_info = fig.add_subplot(gs[:, 3])
    ax_info.set_facecolor(PANEL); ax_info.axis('off')
    ax_info.set_title("Game Tree Facts", color=GOLD, fontsize=11, fontweight='bold')

    facts = [
        ("Board states",      "~5,478"),
        ("Terminal nodes",    "255,168"),
        ("X wins",            "131,184"),
        ("O wins",            "77,904"),
        ("Draws",             "46,080"),
```

```
        ("Tree depth",         "max 9"),
        ("Branching factor",   "≤ 9"),
        ("Result (optimal)",   "Always draw"),
    ]
    for i,(k,v) in enumerate(facts):
        yp = 0.92 - i*0.105
        ax_info.text(0.05, yp, k, transform=ax_info.transAxes,
                     fontsize=9, color=GREY)
        ax_info.text(0.95, yp, v, transform=ax_info.transAxes,
                     fontsize=9, color=WHITE, ha='right', fontweight='bold')
        if i < len(facts)-1:
            line = Line2D([0.04, 0.96], [yp - 0.045, yp - 0.045],
                          color=BORDER, linewidth=0.5,
                          transform=ax_info.transAxes)
            ax_info.add_line(line)

    plt.savefig("outputs-7/fig1_overview.png",
                dpi=150, bbox_inches='tight', facecolor=BG)
    plt.close()
    print("✓ fig1_overview.png")



# ═══════════════════════════════════════════════════════════════
#  FIGURE 2 – MINIMAX SEARCH TREE  (depth-limited, from a mid-game state)
# ═══════════════════════════════════════════════════════════════
```

```
def build_tree_limited(board, max_depth=3):
    """Build tree data up to max_depth levels."""
    tree = []
    minimax(board, 0, True, -math.inf, math.inf, tree, None, None)
    return [n for n in tree if n["depth"] <= max_depth]


def fig_search_tree():
    # A near-terminal board so the tree stays manageable
    board = ['X', 'O', 'X',
             'O', 'X', ' ',
             ' ', ' ', 'O']
    tree = []
    minimax(board, 0, True, -math.inf, math.inf, tree, None, None)

    # Layout: assign x positions per depth level
    by_depth = {}
    for n in tree:
        by_depth.setdefault(n["depth"], []).append(n)

    # BFS-based x assignment
    pos = {}
    for d, nodes in sorted(by_depth.items()):
        n_nodes = len(nodes)
        for i, nd in enumerate(nodes):
```

```
        x = (i + 0.5) / n_nodes
        pos[nd["id"]] = (x, 1.0 - d * 0.18)

fig, ax = plt.subplots(figsize=(20, 11), facecolor=BG)
ax.set_facecolor(BG); ax.axis('off')
fig.suptitle("MINIMAX SEARCH TREE  (Alpha-Beta Pruning)  —  X to Move",
             fontsize=16, fontweight='black', color=WHITE, y=0.98)

# Draw edges first
id_map = {n["id"]: n for n in tree}
for nd in tree:
    if nd["parent"] is not None and nd["parent"] in pos:
        x0,y0 = pos[nd["parent"]]
        x1,y1 = pos[nd["id"]]
        clr = ORANGE if nd.get("pruned") else BORDER
        ax.annotate("", xy=(x1,y1), xytext=(x0,y0),
                    arrowprops=dict(arrowstyle="-|>",
                                    color=clr, lw=0.8,
                                    mutation_scale=8))

# Draw nodes
node_w, node_h = 0.055, 0.13
for nd in tree:
    if nd["id"] not in pos:
        continue
```

```
        cx, cy = pos[nd["id"]]



        # node box colour

        if nd.get("pruned"):

            ec = ORANGE; fc = "#2A1A00"

        elif nd["is_max"]:

            ec = ACCENT_X; fc = "#2A0A0A"

        else:

            ec = ACCENT_O; fc = "#0A2A28"



        rect = FancyBboxPatch((cx - node_w/2, cy - node_h/2),

                              node_w, node_h,

                              boxstyle="round,pad=0.005",

                              facecolor=fc, edgecolor=ec,

                              linewidth=1.2,

                              transform=ax.transData)

        ax.add_patch(rect)



        # tiny board inside node

        b = nd["board"]

        symbols = {' ': '', 'X': 'X', 'O': 'O'}

        colors   = {'X': ACCENT_X, 'O': ACCENT_O, ' ': GREY}

        cell_w = node_w / 3; cell_h = node_h / 3

        for idx in range(9):

            r, c2 = divmod(idx, 3)
```

```
            tx = cx - node_w/2 + (c2 + 0.5) * cell_w
            ty = cy - node_h/2 + (2 - r + 0.5) * cell_h
            # highlight last move
            if nd.get("move") == idx:
                ax.add_patch(Circle((tx, ty), cell_w*0.38,
                                    facecolor="#333300", edgecolor=GOLD,
                                    linewidth=0.5, transform=ax.transData,
                                    zorder=3))
            if b[idx] != ' ':
                ax.text(tx, ty, b[idx],
                        ha='center', va='center', fontsize=5.5,
                        color=colors[b[idx]], fontweight='black',
                        transform=ax.transData, zorder=4)

        # score label
        sc = nd.get("score")
        if sc is not None:
            sc_clr = GREEN if sc > 0 else (ACCENT_O if sc < 0 else GREY)
            ax.text(cx, cy - node_h/2 - 0.025, f"{sc:+d}" if sc != 0 else "0",
                    ha='center', va='top', fontsize=7.5, color=sc_clr,
                    fontweight='bold', transform=ax.transData)

        # MAX/MIN label on left side
        role_txt = "MAX" if nd["is_max"] else "MIN"
        role_clr = ACCENT_X if nd["is_max"] else ACCENT_O
```

```
        if nd["depth"] == 0:
            ax.text(cx - node_w/2 - 0.01, cy,
                    role_txt, ha='right', va='center',
                    fontsize=7, color=role_clr, fontweight='bold',
                    transform=ax.transData)

    # Depth labels on y-axis
    for d, nodes in by_depth.items():
        y = 1.0 - d * 0.18
        if nodes:
            role = "MAX (X)" if nodes[0]["is_max"] else "MIN (O)"
            clr  = ACCENT_X if nodes[0]["is_max"] else ACCENT_O
            ax.text(0.001, y, f"d={d}  {role}",
                    ha='left', va='center', fontsize=8,
                    color=clr, transform=ax.transAxes)

    # Legend
    legend_elements = [
        mpatches.Patch(facecolor="#2A0A0A", edgecolor=ACCENT_X, label="MAX node (X maximises)"),
        mpatches.Patch(facecolor="#0A2A28", edgecolor=ACCENT_O, label="MIN node (O minimises)"),
        mpatches.Patch(facecolor="#2A1A00", edgecolor=ORANGE,   label="Pruned branch"),
        Line2D([0],[0], color=GREEN,   lw=2, label="Score > 0  (X winning)"),
        Line2D([0],[0], color=ACCENT_O,lw=2, label="Score < 0  (O winning)"),
        Line2D([0],[0], color=GREY,    lw=2, label="Score = 0  (Draw)"),
```

```
    ]
    ax.legend(handles=legend_elements, loc='lower center', ncol=3,
              facecolor=PANEL, edgecolor=BORDER, labelcolor=WHITE,
              fontsize=8, bbox_to_anchor=(0.5, -0.01))

    plt.tight_layout()
    plt.savefig("outputs-7/fig2_search_tree.png",
                dpi=150, bbox_inches='tight', facecolor=BG)
    plt.close()
    print("✓ fig2_search_tree.png")


# ═══════════════════════════════════════════════════════════════
#  FIGURE 3 – ALPHA-BETA PRUNING DIAGRAM
# ═══════════════════════════════════════════════════════════════

def fig_alpha_beta():
    """
    Hand-crafted 3-level alpha-beta example tree showing exact
    α/β updates and where pruning fires.
    """
    fig, ax = plt.subplots(figsize=(16, 9), facecolor=BG)
    ax.set_facecolor(BG); ax.axis('off')
    fig.suptitle("ALPHA-BETA PRUNING  —  Step-by-Step Walkthrough",
                 fontsize=16, fontweight='black', color=WHITE, y=0.97)
```

```
# Node definitions: (id, x, y, label, score, is_max, pruned)
nodes = {
    # root
    0:  (0.50, 0.82, "MAX",  None, True,  False),
    # level 1
    1:  (0.20, 0.58, "MIN",  None, False, False),
    2:  (0.50, 0.58, "MIN",  None, False, False),
    3:  (0.80, 0.58, "MIN",  None, False, False),
    # level 2 – leaves
    4:  (0.10, 0.30, "",      3,   True,  False),
    5:  (0.20, 0.30, "",      5,   True,  False),
    6:  (0.30, 0.30, "",      2,   True,  False),
    7:  (0.40, 0.30, "",      9,   True,  False),
    8:  (0.50, 0.30, "",      0,   True,  False),
    9:  (0.60, 0.30, "",      7,   True,  True ),  # PRUNED
    10: (0.70, 0.30, "",      5,   True,  False),
    11: (0.80, 0.30, "",      8,   True,  True ),  # PRUNED
    12: (0.90, 0.30, "",      6,   True,  True ),  # PRUNED
}
# Edges: (parent, child)
edges = [(0,1),(0,2),(0,3),
         (1,4),(1,5),(1,6),
         (2,7),(2,8),(2,9),
         (3,10),(3,11),(3,12)]
```

```
# Computed scores bottom-up
scores = {4:3,5:5,6:2, 7:9,8:0, 10:5}
min_scores = {1: min(3,5,2), 2: min(9,0), 3: 5}  # node 9 pruned
scores.update(min_scores)
scores[0] = max(3,0,5)    # = 5


# Draw edges
for p,c in edges:
    x0,y0,*_ = nodes[p]; x1,y1,*_ = nodes[c]
    pr = nodes[c][5]
    ax.annotate("", xy=(x1,y1+0.045), xytext=(x0,y0-0.055),
                arrowprops=dict(arrowstyle="-|>",
                                color=ORANGE if pr else BORDER,
                                lw=1.5 if pr else 1.0,
                                connectionstyle="arc3,rad=0.0",
                                mutation_scale=10))
    if pr:
        mx = (x0+x1)/2; my = (y0+y1)/2
        ax.text(mx+0.01, my, "✗ PRUNED",
                fontsize=7, color=ORANGE, fontweight='bold')


# Draw nodes
for nid,(x,y,lbl,leaf_sc,is_max,pruned) in nodes.items():
    sc = leaf_sc if leaf_sc is not None else scores.get(nid)
```

```
    ec = ORANGE if pruned else (ACCENT_X if is_max else ACCENT_O)
    fc = "#2A1A00" if pruned else ("#1C0505" if is_max else "#05191C")
    radius = 0.045
    circ = Circle((x,y), radius, facecolor=fc, edgecolor=ec,
                  linewidth=2.0, transform=ax.transData, zorder=3)
    ax.add_patch(circ)

    # role label inside
    if lbl:
        ax.text(x, y+0.01, lbl, ha='center', va='center',
                fontsize=9, color=ec, fontweight='black',
                transform=ax.transData, zorder=4)

    # score outside node
    if sc is not None:
        sc_clr = GREEN if sc>0 else (ACCENT_O if sc<0 else GREY)
        ax.text(x, y - radius - 0.04, str(sc),
                ha='center', va='top', fontsize=11,
                color=sc_clr, fontweight='black',
                transform=ax.transData)

    # leaf: show raw value
    if leaf_sc is not None and not pruned:
        ax.text(x, y, str(leaf_sc), ha='center', va='center',
                fontsize=10, color=WHITE, fontweight='black',
```

```
                transform=ax.transData, zorder=5)



# Alpha-beta annotations
annotations = [
    (0.50, 0.73, "α=3 after left subtree\nβ=+∞", ACCENT_X),
    (0.20, 0.46, "min(3,5,2)=3\n→ parent α=3", ACCENT_O),
    (0.50, 0.46, "min(9,0)=0 < α=3\nNode 9 pruned!", ORANGE),
    (0.80, 0.46, "5 ≥ α=3  →  no pruning\n11,12 still pruned (β≤α)", ORANGE),
]
for ax_,ay_,txt,clr in annotations:
    ax.text(ax_, ay_-0.06, txt, ha='center', va='top',
            fontsize=7.5, color=clr, linespacing=1.4,
            bbox=dict(boxstyle='round,pad=0.3', facecolor=PANEL,
                      edgecolor=clr, linewidth=0.8),
            transform=ax.transAxes)



# Step legend
steps_txt = (
    "STEP-BY-STEP:\n"
    "① Evaluate leftmost subtree (leaves 4,5,6) → MIN node 1 = 3\n"
    "② Root MAX updates α = max(-∞, 3) = 3\n"
    "③ Evaluate subtree of node 2: leaf 7=9, leaf 8=0 → partial min=0\n"
    "   β(node 2) = 0  ≤  α(root) = 3  →  PRUNE leaf 9\n"
    "④ Evaluate subtree of node 3: leaf 10=5\n"
    "   β(node 3) = 5  ≥  α=3  →  continue  (but leaves 11,12 ≤ 5 → pruned)\n"
```

```
        "⑤ Root MAX = max(3, 0, 5) = 5  →  choose rightmost branch"
    )
    ax.text(0.02, 0.15, steps_txt, transform=ax.transAxes,
            fontsize=9, color=WHITE, linespacing=1.6, va='top',
            bbox=dict(boxstyle='round,pad=0.5', facecolor=PANEL,
                      edgecolor=PURPLE, linewidth=1.2))

    # Level labels
    for y_, txt, clr in [(0.82,"Level 0 — MAX (root)", ACCENT_X),
                         (0.58,"Level 1 — MIN",        ACCENT_O),
                         (0.30,"Level 2 — Leaves",     GOLD)]:
        ax.text(0.01, y_, txt, transform=ax.transAxes,
                fontsize=8, color=clr, va='center', fontweight='bold')

    plt.tight_layout()
    plt.savefig("outputs-7/fig3_alpha_beta.png",
                dpi=150, bbox_inches='tight', facecolor=BG)
    plt.close()
    print("✓ fig3_alpha_beta.png")


# ═══════════════════════════════════════════════════════════════
#  FIGURE 4 – FULL GAME SIMULATION  (AI vs AI)
# ═══════════════════════════════════════════════════════════════
```

```python
def simulate_game():
    """Simulate an AI-vs-AI game, returns list of (board_snapshot, move, player)."""
    board = [' ']*9
    history = [(board[:], None, None)]
    turn = 0
    while not is_terminal(board):
        player = 'X' if turn % 2 == 0 else 'O'
        is_max = (player == 'X')
        # For O (minimiser) we negate
        if is_max:
            mv, _ = best_move(board)
        else:
            # Best move for O
            best_sc = math.inf; mv = None
            for m in get_available_moves(board):
                b2 = board[:]; b2[m] = 'O'
                s = minimax(b2, 0, True)
                if s < best_sc:
                    best_sc, mv = s, m
        board[mv] = player
        history.append((board[:], mv, player))
        turn += 1
    return history


def fig_game_simulation():
```

```
history = simulate_game()
n = len(history)   # typically 6-9 steps + initial = up to 10 frames

cols = 5
rows = math.ceil(n / cols)
fig = plt.figure(figsize=(cols*3.2, rows*3.5), facecolor=BG)
fig.suptitle("FULL GAME SIMULATION  —  Minimax X  vs  Minimax O  (Optimal Play)",
             fontsize=14, fontweight='black', color=WHITE, y=0.99)

for i, (board, move, player) in enumerate(history):
    ax = fig.add_subplot(rows, cols, i+1)
    winner, wl = check_winner(board)
    hl = [move] if move is not None else []
    title_clr = GOLD if i==0 else (ACCENT_X if player=='X' else ACCENT_O)
    if winner:
        title_clr = GREEN
    title = ("Initial Board" if i==0
             else f"Move {i}: {player} → cell {move}"
             + (" ✓ WINS!" if winner else ""))
    if i == len(history)-1 and not winner:
        title = f"Move {i}: {player} → cell {move}   · DRAW ·"
        title_clr = GREY

    draw_board_on_ax(ax, board,
                     highlight_cells=hl,
```

```
                        win_line=list(wl) if wl else None,
                        title=title, title_color=title_clr,
                        fontsize=8)

    plt.tight_layout(rect=[0, 0, 1, 0.97])
    plt.savefig("outputs-7/fig4_game_simulation.png",
                dpi=150, bbox_inches='tight', facecolor=BG)
    plt.close()
    print("✓ fig4_game_simulation.png")



# ═══════════════════════════════════════════════════════════
#  FIGURE 5 – HEATMAP: best-move frequency from all opening positions
# ═══════════════════════════════════════════════════════════

def fig_heatmap():
    """Show how often each cell is chosen as the best first move."""
    counts = np.zeros(9)
    # From the empty board, X always picks the same cell, but let's show
    # best moves from all single-O boards (second ply)
    for o_pos in range(9):
        board = [' ']*9
        board[o_pos] = 'O'
        mv, _ = best_move(board)
        counts[mv] += 1
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5.5), facecolor=BG)
fig.suptitle("BEST-MOVE HEATMAP  —  X's Optimal Response to Every O Opening",
             fontsize=13, fontweight='black', color=WHITE, y=1.01)

# Left: heatmap grid
ax = axes[0]; ax.set_facecolor(BG); ax.axis('off')
ax.set_title("Frequency X chooses each cell\n(across all 9 O opening moves)",
             color=GOLD, fontsize=10)
cmap = plt.cm.YlOrRd
norm_counts = counts / counts.max()
for idx in range(9):
    row, col = divmod(idx, 3)
    y = 2 - row; x = col
    clr = cmap(norm_counts[idx])
    rect = FancyBboxPatch((x+0.05, y+0.05), 0.9, 0.9,
                          boxstyle="round,pad=0.05",
                          facecolor=clr, edgecolor=WHITE, linewidth=0.8)
    ax.add_patch(rect)
    ax.text(x+0.5, y+0.55, str(int(counts[idx])),
            ha='center', va='center', fontsize=20,
            color='white' if norm_counts[idx]>0.4 else BG,
            fontweight='black')
    ax.text(x+0.5, y+0.20, f"cell {idx}",
            ha='center', va='center', fontsize=8,
```

```
                color='white' if norm_counts[idx]>0.4 else BG)

    ax.set_xlim(0,3); ax.set_ylim(0,3); ax.set_aspect('equal')



    # Colourbar

    sm = plt.cm.ScalarMappable(cmap=cmap,

                               norm=plt.Normalize(0, counts.max()))

    sm.set_array([])

    cb = plt.colorbar(sm, ax=ax, fraction=0.03, pad=0.04)

    cb.set_label("Times chosen", color=WHITE, fontsize=8)

    cb.ax.yaxis.set_tick_params(color=WHITE)

    plt.setp(cb.ax.yaxis.get_ticklabels(), color=WHITE)



    # Right: bar chart

    ax2 = axes[1]; ax2.set_facecolor(PANEL)

    cell_labels = [f"Cell {i}" for i in range(9)]

    bar_colors = [cmap(norm_counts[i]) for i in range(9)]

    bars = ax2.bar(cell_labels, counts, color=bar_colors, edgecolor=BORDER,
linewidth=0.8)

    ax2.set_facecolor(PANEL)

    ax2.set_title("Bar Chart of Best-Move Frequency", color=GOLD, fontsize=10)

    ax2.set_xlabel("Cell Index (0-8)", color=GREY, fontsize=9)

    ax2.set_ylabel("Frequency", color=GREY, fontsize=9)

    ax2.tick_params(colors=WHITE, rotation=30)

    ax2.spines[:].set_color(BORDER)

    for spine in ax2.spines.values():
```

```
        spine.set_edgecolor(BORDER)
    for bar, cnt in zip(bars, counts):
        if cnt:
            ax2.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.05,
                     str(int(cnt)), ha='center', va='bottom',
                     color=WHITE, fontsize=9, fontweight='bold')

    ax2.set_ylim(0, counts.max()*1.25)
    ax2.text(0.98, 0.97,
             "Cell 4 (centre) is\nX's most common reply",
             transform=ax2.transAxes, ha='right', va='top',
             fontsize=8.5, color=GOLD,
             bbox=dict(boxstyle='round', facecolor=BG, edgecolor=GOLD, lw=1))

    plt.tight_layout()
    plt.savefig("outputs-7/fig5_heatmap.png",
                dpi=150, bbox_inches='tight', facecolor=BG)
    plt.close()
    print("✓ fig5_heatmap.png")


# ═══════════════════════════════════════════════════════════
#  FIGURE 6 — NODE COUNT: Minimax vs Alpha-Beta comparison
# ═══════════════════════════════════════════════════════════
```

```
_node_counter = [0]



def minimax_count(board, depth, is_max, counter, alpha=-math.inf, beta=math.inf,
ab=True):

    counter[0] += 1

    winner, _ = check_winner(board)

    if winner == 'X':  return 10-depth

    if winner == 'O':  return depth-10

    if ' ' not in board: return 0



    moves = get_available_moves(board)

    best = -math.inf if is_max else math.inf

    for mv in moves:

        b2 = board[:]; b2[mv] = 'X' if is_max else 'O'

        s = minimax_count(b2, depth+1, not is_max, counter, alpha, beta, ab)

        if is_max:

            best = max(best,s)

            if ab: alpha = max(alpha,s)

        else:

            best = min(best,s)

            if ab: beta = min(beta,s)

        if ab and beta<=alpha: break

    return best



def fig_node_comparison():
```

```
    boards = [
        [' ']*9,
        ['X',' ',' ',' ',' ',' ',' ',' ',' '],
        ['X','O',' ',' ','X',' ',' ',' ',' '],
        ['X','O','X','O','X',' ',' ',' ','O'],
    ]
    labels = ["Empty\n(0 moves)", "1 move\nplayed", "4 moves\nplayed", "6
moves\nplayed"]


    plain_counts = []
    ab_counts    = []
    for b in boards:
        c1=[0]; c2=[0]
        minimax_count(b,0,True,c1,ab=False)
        minimax_count(b,0,True,c2,ab=True)
        plain_counts.append(c1[0])
        ab_counts.append(c2[0])


    x = np.arange(len(labels))
    w = 0.35


    fig, axes = plt.subplots(1, 2, figsize=(13, 5.5), facecolor=BG)
    fig.suptitle("NODES EVALUATED: Plain Minimax vs Alpha-Beta Pruning",
                 fontsize=13, fontweight='black', color=WHITE, y=1.01)
```

```
# Bar chart
ax = axes[0]; ax.set_facecolor(PANEL)
ax.bar(x-w/2, plain_counts, width=w, color=ACCENT_X, edgecolor=BG,
       label="Plain Minimax")
ax.bar(x+w/2, ab_counts,    width=w, color=ACCENT_O, edgecolor=BG,
       label="Alpha-Beta")
for i,(p,a) in enumerate(zip(plain_counts,ab_counts)):
    ax.text(i-w/2, p+100, f"{p:,}", ha='center', fontsize=7.5,
            color=ACCENT_X, fontweight='bold')
    ax.text(i+w/2, a+100, f"{a:,}", ha='center', fontsize=7.5,
            color=ACCENT_O, fontweight='bold')
ax.set_xticks(x); ax.set_xticklabels(labels, color=WHITE, fontsize=9)
ax.set_ylabel("Nodes evaluated", color=GREY, fontsize=9)
ax.set_title("Node Count Comparison", color=GOLD, fontsize=10)
ax.legend(facecolor=BG, labelcolor=WHITE, edgecolor=BORDER)
ax.tick_params(colors=WHITE)
ax.spines[:].set_color(BORDER)
ax.set_facecolor(PANEL)


# Reduction % line chart
ax2 = axes[1]; ax2.set_facecolor(PANEL)
reduction = [(1 - a/p)*100 if p else 0 for p,a in zip(plain_counts, ab_counts)]
ax2.plot(labels, reduction, color=GOLD, lw=2.5, marker='o', markersize=9,
         markerfacecolor=BG, markeredgecolor=GOLD, markeredgewidth=2)
for i,(lbl,r) in enumerate(zip(labels, reduction)):
```

```
        ax2.text(i, r+1.5, f"{r:.0f}%", ha='center', fontsize=9,
                 color=GOLD, fontweight='bold')
    ax2.set_ylim(0, 100); ax2.set_xlabel("Board State", color=GREY, fontsize=9)
    ax2.set_ylabel("Nodes Pruned (%)", color=GREY, fontsize=9)
    ax2.set_title("Pruning Efficiency (%)", color=GOLD, fontsize=10)
    ax2.tick_params(colors=WHITE)
    ax2.spines[:].set_color(BORDER)
    ax2.axhline(50, color=BORDER, lw=0.8, linestyle='--')
    ax2.text(3.05, 50, "50%", color=GREY, fontsize=8, va='center')

    plt.tight_layout()
    plt.savefig("outputs-7/fig6_node_comparison.png",
                dpi=150, bbox_inches='tight', facecolor=BG)
    plt.close()
    print("✓ fig6_node_comparison.png")


# ═══════════════════════════════════════════════════════════
#  FIGURE 7 – SCORE LANDSCAPE  (all possible first moves)
# ═══════════════════════════════════════════════════════════

def fig_score_landscape():
    fig, axes = plt.subplots(1, 3, figsize=(14, 5), facecolor=BG)
    fig.suptitle("MINIMAX SCORE LANDSCAPE  —  X's Score for Every Possible Move",
                 fontsize=13, fontweight='black', color=WHITE, y=1.01)
```

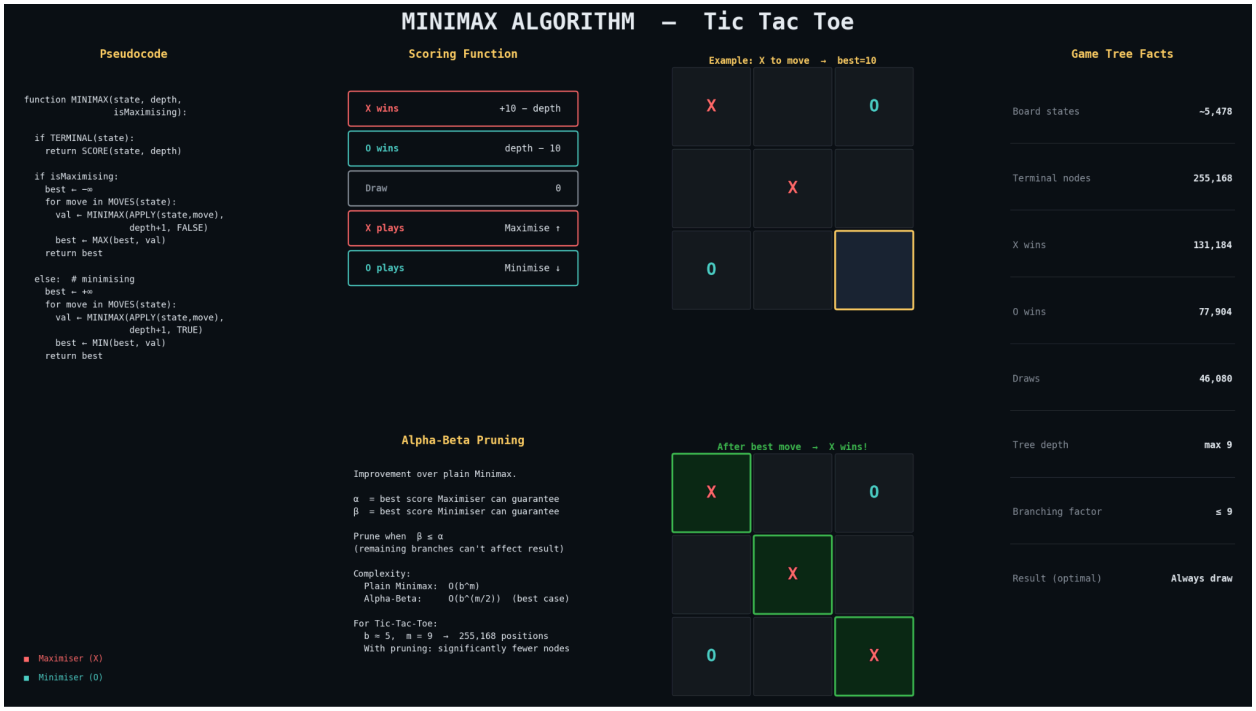
```
scenarios = [
    ([' ']*9,   True,  "Empty Board — X first"),
    (['O',' ',' ',' ',' ',' ',' ',' ',' '], True, "After O→0"),
    (['O',' ',' ',' ','X',' ',' ',' ','O'], True, "After O→0,X→4,O→8"),
]

for ax, (board, is_max, title) in zip(axes, scenarios):
    ax.set_facecolor(BG); ax.axis('off')
    ax.set_title(title, color=GOLD, fontsize=9, fontweight='bold')
    scores_grid = np.full(9, np.nan)
    for mv in get_available_moves(board):
        b2 = board[:]; b2[mv] = 'X' if is_max else 'O'
        s = minimax(b2, 0, not is_max)
        scores_grid[mv] = s

    cmap = plt.cm.RdYlGn
    finite = scores_grid[~np.isnan(scores_grid)]
    vmin = finite.min() if len(finite) else -10
    vmax = finite.max() if len(finite) else 10

    for idx in range(9):
        row, col = divmod(idx, 3)
        y = 2-row; x = col
        val = scores_grid[idx]
```

```
            if np.isnan(val):
                # occupied cell
                fc = "#0A0A0A"; txt = board[idx]
                txt_clr = ACCENT_X if txt=='X' else ACCENT_O
                fs = 18
            else:
                norm_v = (val-vmin)/(vmax-vmin+1e-9)
                fc = cmap(norm_v)
                txt = f"{val:+.0f}" if val != 0 else "0"
                txt_clr = BG
                fs = 14

            rect = FancyBboxPatch((x+0.05, y+0.05), 0.9, 0.9,
                                  boxstyle="round,pad=0.05",
                                  facecolor=fc, edgecolor=WHITE, linewidth=0.6)
            ax.add_patch(rect)
            ax.text(x+0.5, y+0.5, txt,
                    ha='center', va='center', fontsize=fs,
                    color=txt_clr, fontweight='black')

            # best move star
            if not np.isnan(val) and len(finite) and val == finite.max():
                ax.text(x+0.82, y+0.82, "★",
                        ha='center', va='center', fontsize=11, color=GOLD)
```

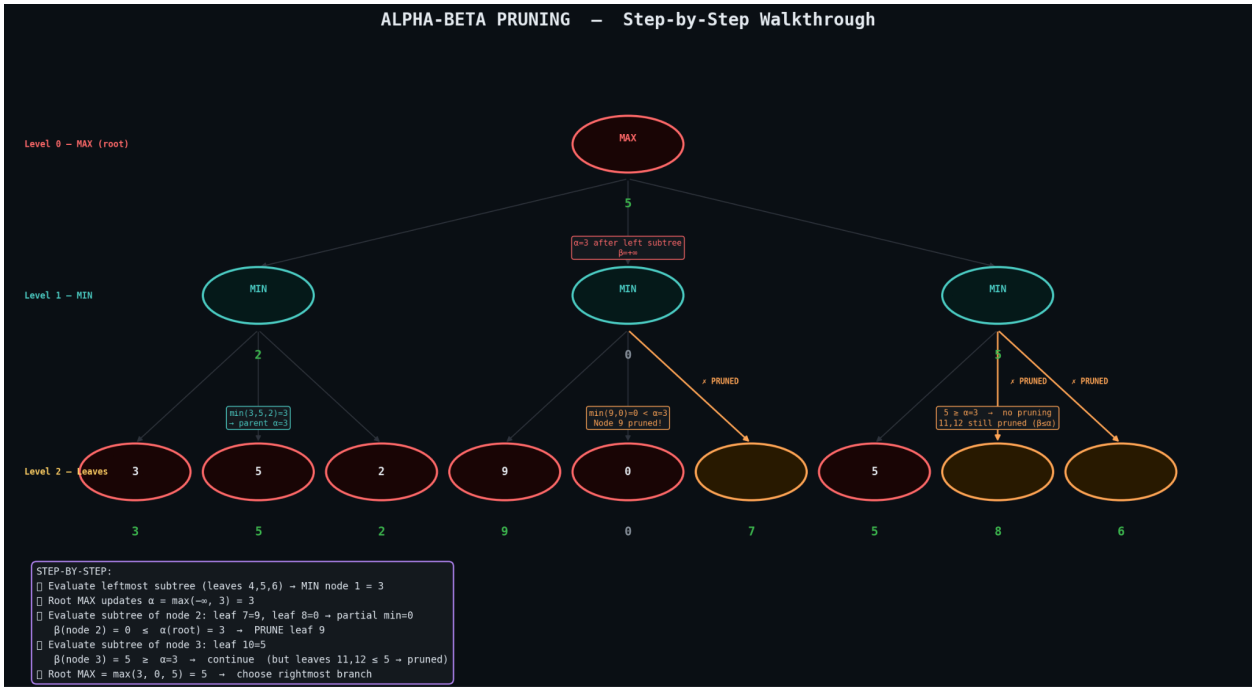
```
        ax.set_xlim(0,3); ax.set_ylim(0,3); ax.set_aspect('equal')


    # Shared colorbar
    sm2 = plt.cm.ScalarMappable(cmap=plt.cm.RdYlGn,
                                norm=plt.Normalize(-10,10))
    sm2.set_array([])
    cb = plt.colorbar(sm2, ax=axes, fraction=0.015, pad=0.03)
    cb.set_label("Minimax Score", color=WHITE, fontsize=9)
    cb.ax.yaxis.set_tick_params(color=WHITE)
    plt.setp(cb.ax.yaxis.get_ticklabels(), color=WHITE)


    plt.tight_layout()
    plt.savefig("outputs-7/fig7_score_landscape.png",
                dpi=150, bbox_inches='tight', facecolor=BG)
    plt.close()
    print("✓ fig7_score_landscape.png")



# ═══════════════════════════════════════════════════════════
#  MAIN
# ═══════════════════════════════════════════════════════════

if __name__ == "__main__":
    print("Generating all figures...")
    fig_overview()
```

```
fig_search_tree()

fig_alpha_beta()

fig_game_simulation()

fig_heatmap()

fig_node_comparison()

fig_score_landscape()

print("\n✅  All 7 figures saved to outputs-7/")
```

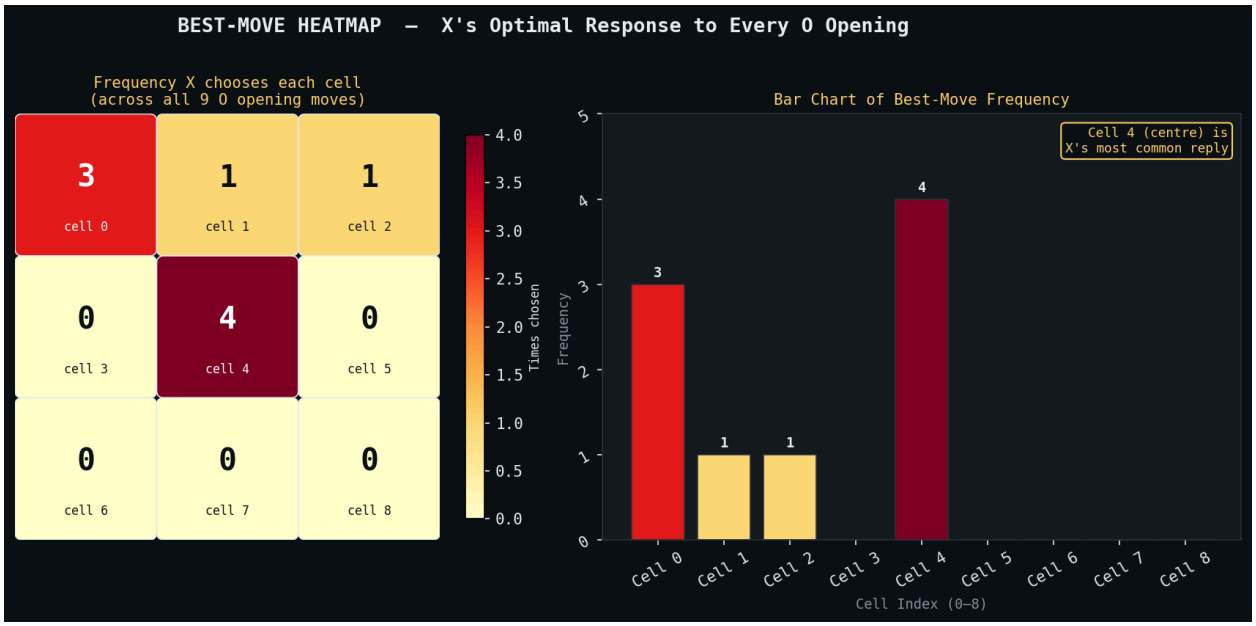
Output:

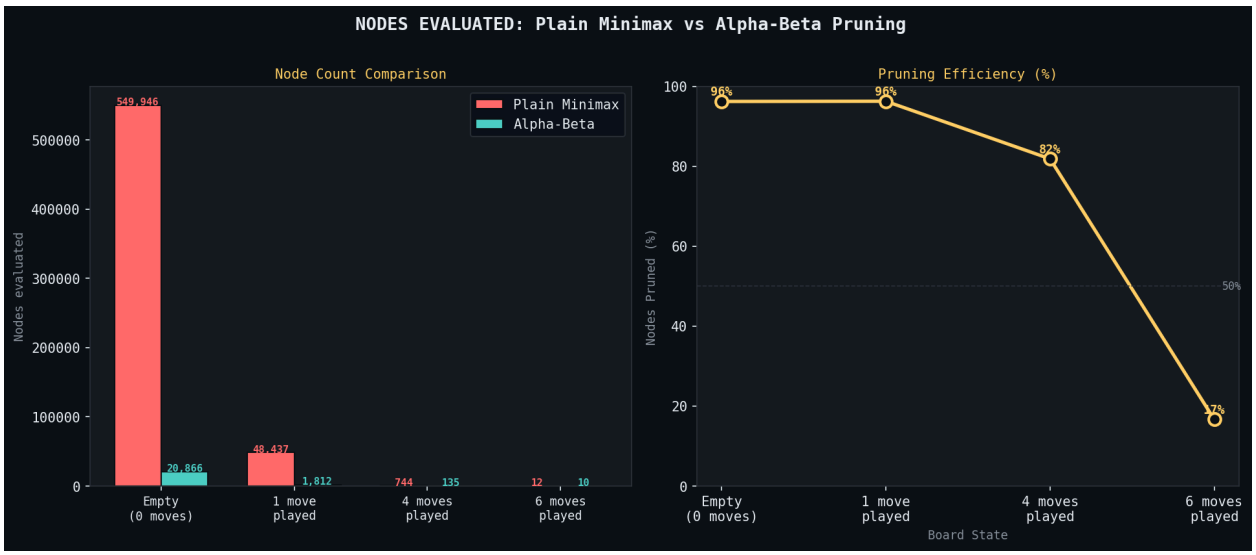


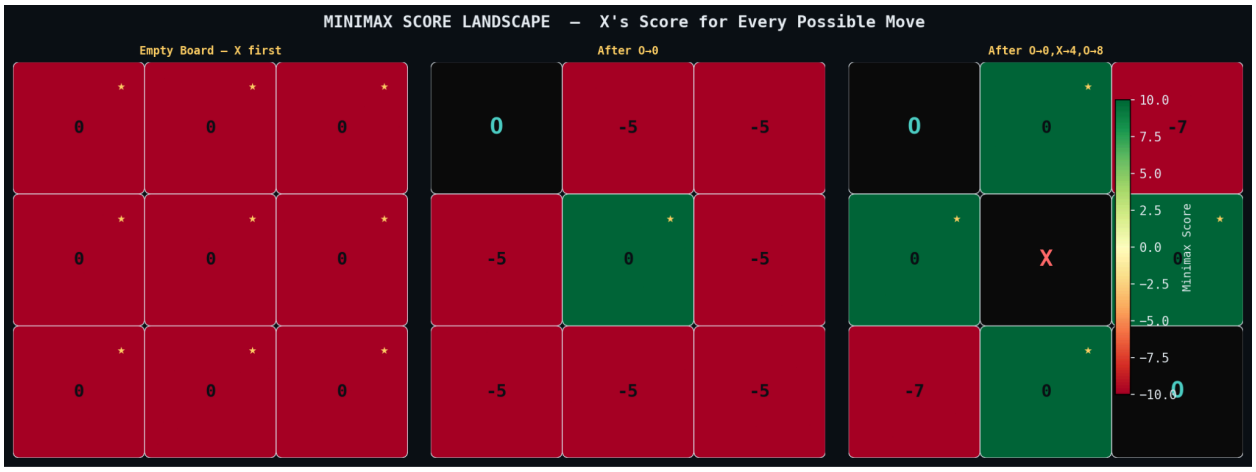

MINIMAX SEARCH TREE (Alpha-Beta Pruning) — X to Move
d=0 MAX (X)
d=1 MIN (O)
d=2 MAX (X)
d=3 MIN (O)
MAX
MAX node (X maximises)
MIN node (O minimises)
Pruned branch
Score > 0 (X winning)
Score < 0 (O winning)
Score = 0 (Draw)


ALPHA-BETA PRUNING — Step-by-Step Walkthrough
Level 0 — MAX (root)
Level 1 — MIN
Level 2 — Leaves
MAX
MIN
MIN
MIN
5
α=3 after left subtree
β=+∞
2
0
5
✗ PRUNED
✗ PRUNED
✗ PRUNED
min(3,5,2)=3
→ parent α=3
min(9,0)=0 < α=3
Node 9 pruned!
5 ≥ α=3 → no pruning
11,12 still pruned (β≤α)
3
5
2
9
0
5
3
5
2
9
0
7
5
8
6
STEP-BY-STEP:
① Evaluate leftmost subtree (leaves 4,5,6) → MIN node 1 = 3
② Root MAX updates α = max(−∞, 3) = 3
③ Evaluate subtree of node 2: leaf 7=9, leaf 8=0 → partial min=0
β(node 2) = 0 ≤ α(root) = 3 → PRUNE leaf 9
④ Evaluate subtree of node 3: leaf 10=5
β(node 3) = 5 ≥ α=3 → continue (but leaves 11,12 ≤ 5 → pruned)
⑤ Root MAX = max(3, 0, 5) = 5 → choose rightmost branch


FULL GAME SIMULATION — Minimax X vs Minimax O (Optimal Play)
Initial Board
Move 1: X → cell 0
Move 2: O → cell 4
Move 3: X → cell 1
Move 4: O → cell 2
Move 5: X → cell 6
Move 6: O → cell 3
Move 7: X → cell 5
Move 8: O → cell 7
Move 9: X → cell 8 · DRAW ·


BEST-MOVE HEATMAP — X's Optimal Response to Every O Opening
Frequency X chooses each cell
(across all 9 O opening moves)
3
cell 0
1
cell 1
1
cell 2
0
cell 3
4
cell 4
0
cell 5
0
cell 6
0
cell 7
0
cell 8
Times chosen
Bar Chart of Best-Move Frequency
Cell 4 (centre) is
X's most common reply
Frequency
Cell Index (0–8)
Cell 0
Cell 1
Cell 2
Cell 3
Cell 4
Cell 5
Cell 6
Cell 7
Cell 8


NODES EVALUATED: Plain Minimax vs Alpha-Beta Pruning
Node Count Comparison
Plain Minimax
Alpha-Beta
Nodes evaluated
549,946
20,866
48,437
1,812
744
135
12
10
Empty
(0 moves)
1 move
played
4 moves
played
6 moves
played
Pruning Efficiency (%)
Nodes Pruned (%)
96%
96%
82%
17%
50%
Board State


MINIMAX SCORE LANDSCAPE — X's Score for Every Possible Move
Empty Board — X first
After O→0
After O→0,X→4,O→8
0
0
0
0
0
0
0
0
0
0
-5
-5
-5
0
-5
-5
-5
-5
0
0
0
X
-7
0
-7
0
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
Minimax Score